Testing with JUnit

In the previous example we wrote a Vector class, and a program to test its methods. We will continue with the same example; you will need the code for the Vector class.

The VectorTest program we wrote is helpful because it can do our tests automatically. However, it took a lot of duplicated code to write. Since writing automated tests is so important, we can save a lot of work by using a **testing framework** which handles the boring parts. We'll use **JUnit**.

Here's what a JUnit test class looks like:

```
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class VectorTest {
   @Test
   public void testGetX() {
      Vector vector = new Vector(3, 4);
      assertEquals(3, vector.getX());
   }

   @Test
   public void testGetY() {
      Vector vector = new Vector(3, 4);
      assertEquals(4, vector.getY());
   }

   @Test
   public void testLength() {
      Vector vector = new Vector(3, 4);
      // error tolerance of 0.0 - result must be exact
      assertEquals(5.0, vector.length(), 0.0);
   }
}
```

There are a few new things to explain in this code:

- The import declarations at the top allow us to use Test and assertEquals from the JUnit "package". The assertEquals import is static since it's a static method.
- The keyword public allows JUnit to access our code. Since JUnit itself is in a separate "package", it can only see our class and methods if we make them "public".
- JUnit uses the *@Test* annotation to know which methods are tests.

Assertions in JUnit

Every test should make at least one **assertion** about what the result of the test should be. In the VectorTest class above, the assertEquals method is used three times:

- To assert that 3 is returned by the getX method.
- To assert that 4 is returned by the getY method.
- To assert that 5.0 is returned by the length method.

The assertEquals method checks whether the expected value (first argument) equals the actual value (second argument). If they are not equal, the test will fail.

When comparing double values, a third argument is needed for the error tolerance. In this example, the error tolerance is 0.0 because we want the result to be exactly 5.0. However, since double has a limited precision, calculations often don't give exact results, so usually we should allow a small error. For example, an error tolerance of 0.5e-10 means the result must be accurate to 10 decimal places:

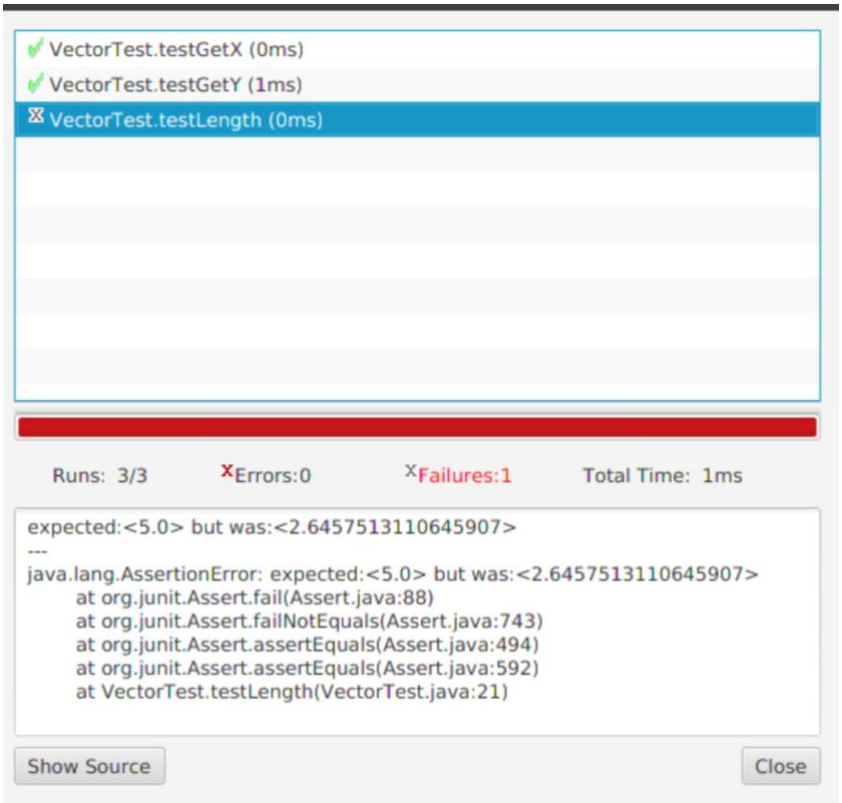
```
@Test
public void testLengthImprecise() {
   Vector vector = new Vector(1, 7);
   // result must be accurate to 10 decimal places
   assertEquals(7.0710678119, vector.length(), 0.5e-10);
}
```

Some other useful JUnit assertion methods are:

- assertTrue tests that some condition is true.
- assertFalse tests that some condition is false.
- assertNull tests that some reference is null.
- assertNotNull tests that some reference is not null.
- assertNotEquals tests that two values are different.
- fail causes the test to fail unconditionally. This should be used to test that a particular line of code will not be reached.

Running JUnit tests

To run the tests from Eclipse, simply right-click on the `VectorTest` class and select “Run as” from the context menu. This runs the tests, then shows the results — and some statistics — in a window:



From here we can see that the first two tests passed, but the test of the `length` method failed (just as it did when we tested manually).